Project Plan

✅ 1. High-Level Architecture

Frontend (React)
↓
API Layer (Axios/Fetch)
↓
Backend (Node + Express)
↓
RAG Layer (LangChain)
↓
Knowledge Base (Pre-chunked JSON)

✅ 2. Project Folder Structure

Root Structure

```
college-helpdesk-agent/
│
├── frontend/
├── backend/
├── README.md
```

Frontend (React)

```
frontend/
│
├── public/
│
├── src/
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Navbar.jsx
│   │   │   └── Footer.jsx
│   │   │
│   │   ├── sections/
│   │   │   ├── HeroSection.jsx
│   │   │   ├── ChatSection.jsx
│   │   │   ├── CoursesSection.jsx
```

```
│   │   │   ├── AdmissionsSection.jsx
│   │   │   └── PlacementSection.jsx
│   │   │
│   │   ├── chatbot/
│   │   │   ├── ChatDrawer.jsx
│   │   │   ├── MessageBubble.jsx
│   │   │   └── ChatInput.jsx
│   │
│   ├── services/
│   │   └── api.js
│   │
│   ├── data/
│   │   ├── courses.json
│   │   ├── admissions.json
│   │   └── placements.json
│   │
│   ├── App.jsx
│   ├── main.jsx
│   └── styles/
│       └── global.css
│
├── package.json
```

Backend (Node + Express + RAG)

```
backend/
│
├── src/
│   ├── controllers/
│   │   └── chatController.js
│   │
│   ├── routes/
│   │   └── chatRoutes.js
│   │
│   ├── services/
│   │   ├── ragService.js
│   │   ├── embeddingService.js
│   │   └── retrievalService.js
│   │
│   ├── utils/
│   │   └── formatter.js
│   │
│   ├── data/
│   │   └── knowledgeBase.json   ✅ (Pre-chunked data)
│   │
│   ├── config/
│   │   └── langchain.js
```

```
│   │
│   ├── app.js
│   └── server.js
│
├── package.json
```

✅ 3. Knowledge Base Structure (FINALIZED)

backend/src/data/knowledgeBase.json

Example:

```
[
 {
  "id": 1,
  "text": "B.Tech Computer Science course duration is 4 years and includes subjects like AI, ML, and Data Structures.",
  "type": "course"
 },
 {
  "id": 2,
  "text": "The average placement package is 6 LPA and highest is 18 LPA.",
  "type": "placement"
 },
 {
  "id": 3,
  "text": "Admission requires 12th pass with minimum 60% and entrance exam qualification.",
  "type": "admission"
 }
]
```

✅ 4. Backend Architecture (RAG Pipeline)

Flow

User Query
↓
Embedding Generation

↓
Similarity Search
↓
Top K Results
↓
Formatter
↓
Response

Core Services

1. embeddingService.js

- Converts text → vector
- Uses Google GenAI embeddings

---

2. retrievalService.js

- Compares query embedding with stored embeddings
- Returns top 3–5 relevant chunks

---

3. ragService.js

- Orchestrates:
  - embedding
  - retrieval
  - formatting

---

4. formatter.js

Formats retrieved chunks into readable response:

```
export const formatResponse = (chunks) => {
 return chunks.map(c => `• ${c.text}`).join("\n");
};
```

✅ 5. API Design

POST /api/chat

Request:

{
 "message": "What is the placement package?"
}

Response:

{
 "reply": "• The average placement package is 6 LPA...\n• Highest package is 18 LPA..."
}

✅ 6. Frontend Architecture

Main Flow

User Input → API Call → Response → UI Update

Chat Flow

- `ChatInput` → send message
- `ChatDrawer` → manage state
- `MessageBubble` → display messages

Sections (Homepage)

All rendered inside `App.jsx`:

<Navbar />
<HeroSection />
<ChatSection />
<CoursesSection />
<AdmissionsSection />
<PlacementSection />
<Footer />

✅ 7. Step-by-Step Implementation Plan

Phase 1: Setup

Backend

- Initialize Node project
- Install:
  - express
  - cors
  - dotenv
  - langchain
  - @google/generative-ai

Frontend

- Create React app (Vite recommended)
- Install axios

Phase 2: Data Layer

- Create `knowledgeBase.json`
- Add 20–30 realistic chunks

Phase 3: RAG Backend

1. Implement embedding service
2. Load JSON data
3. Precompute embeddings (on server start)
4. Implement similarity search
5. Add formatter

Phase 4: API

- Create `/api/chat`
- Connect to ragService

Phase 5: Frontend UI

Build:

- Navbar
- Sections (Courses, Admissions, Placement)
- Chat UI

Phase 6: Integration

- Connect frontend → backend
- Handle loading + errors

Phase 7: Testing

- Empty input
- No result
- API failure

Phase 8: Polish

- Responsive design
- Smooth scrolling
- Better UI styling

✅ 8. Key Design Decisions (Why this works)

- Pre-chunked data → simpler RAG
- No DB → faster development
- Formatter → readable output without generation
- Single-page → matches SRS exactly

✅ 9. Optional Enhancements (Future)

- Add vector DB (Pinecone / FAISS)
- Add real LLM generation
- Add admin dashboard
- Multi-college support